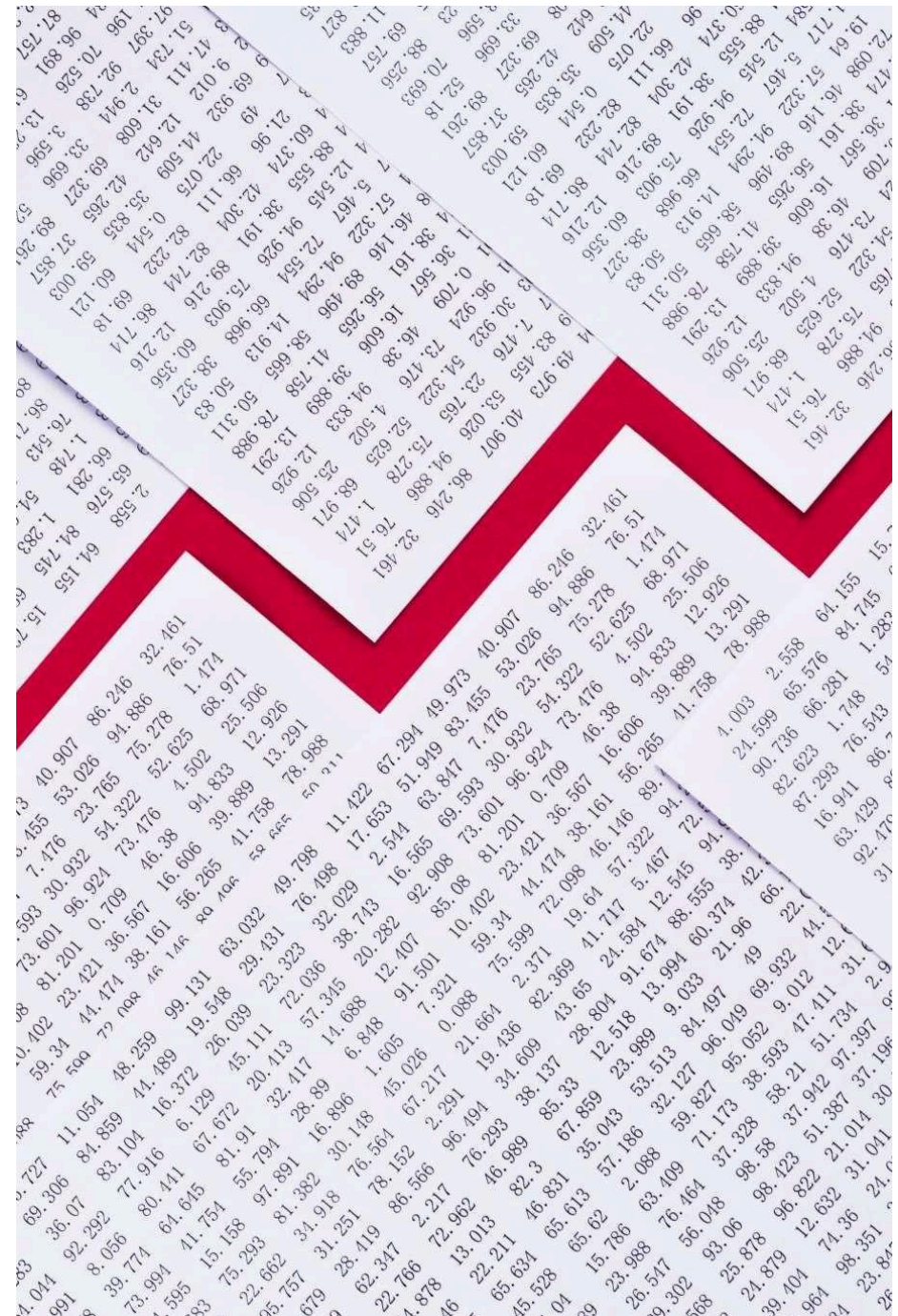


Module 1 : Database System Concepts and Architecture

Syllabus: Introduction, Characteristics of Databases, File system v/ s Database system, Data abstraction and Data Independence, DBMS system architecture, Database Administrator (DBA), Role of DBA. **04 hours**

An overview of Database Management System

- **What is data?**
- Data is the known facts or figures that can be recorded.
- It can also be defined as it is the representation of facts ,concepts or instruction in a formal manner, which is suitable for understanding and processing.
- Data can be represented in alphabets(A-Z, a-z),in digits(0-9) and using special characters(+,-.#,\$, etc) e.g: 25, “ajit” etc.
- Raw Data or Unprocessed data



An overview of Database Management System

- **What is Database?**
- Collection of related data that represents some real world entity.
- Example: Marks of 10000 Students
- **What is a Database Management System?**
- **A Database Management System (DBMS) is software designed to store, retrieve, define, and manage data in a database.**

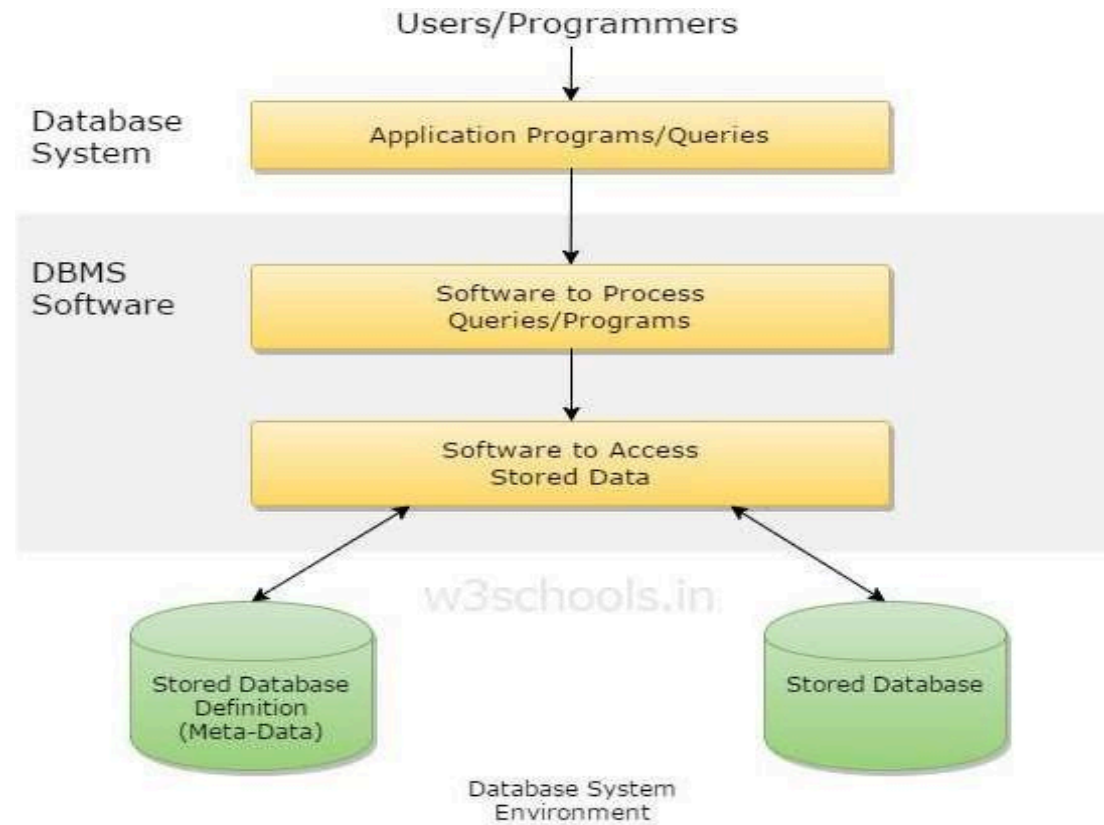


An overview of Database Management System

A **database-management system** (DBMS) is a collection of interrelated data and a set of programs to access those data. The collection of data, usually referred to as the **database**.

(or)

The DBMS is a general-purpose software system that facilitates the processes of defining, constructing, manipulating, and sharing databases among various users and applications.



Database System Applications

- ✓ *Banking*
- ✓ *Airlines*
- ✓ *Universities*
- ✓ *Credit card transactions*
- ✓ *Finance*

✓ *Sales*

✓ *Manufacturing*

✓ *Human resources*

Advantages of Database Systems

1. Data Sharing

- Multiple users and applications can access the same data at the same time.
- No need to duplicate data manually.
- *Example:* A bank teller and an ATM can access the same account simultaneously.

2. Reduced Data Redundancy (Avoid Duplication)

- Data is stored centrally, so the same data is not repeated in multiple places.
- Saves storage and avoids errors.
- *Example:* A customer's address is stored once, not repeated in every bill.

3. Data Consistency

- Since redundancy is reduced, data remains uniform and accurate

Advantages of Database Systems

across the system.

- *Example:* If a student changes their phone number, it updates

Everywhere.

4. Improved Data Security

- Only authorized users can access or modify data using authentication & permissions.
- Supports encryption, access controls, and auditing.
- *Example:* Admin can view all data; student can view only their own results.

5. Data Integrity

- DBMS enforces rules (constraints) like primary key, foreign key, check, etc.
- Ensures correct and valid data.
- *Example:* Age cannot be negative; roll number must be unique.

Advantages of Database Systems

-

6. Backup and Recovery

- Databases support automatic backup and restoration.
- Helps recover data during system failures.

7. Concurrent Access

- Many users can read/write data simultaneously without conflicts.
- Uses locking and transaction management.
- *Example:* Online ticket booking handled for thousands of users.

8. Query Processing & Better Decision Making

- Data can be processed quickly using SQL queries.
- Helps in analytics and reporting.
- *Example:* Sales trends, employee performance, inventory analysis.

9. Scalability

- Database systems can handle increasing data volume and users

Advantages of Database Systems

efficiently.

10. Data Independence

- Data storage and applications are separated.
- Application doesn't need to change if data storage changes.

11. Standardization

- DBMS follows ANSI SQL standards.
- Makes development, maintenance, and migration easier.

Applications of Database Systems

1. Banking and Financial Systems

- Customer accounts, ATM transactions, loans, fund transfers, online banking
- Database stores account balances and transaction history
- *Examples:*
 - ✓ SBI Core Banking System
 - ✓ ATM Networks
 - ✓ UPI transactions

2. Railway / Airline Reservation Systems

- Ticket booking, availability, cancellation, passenger records
- Requires real-time concurrent access
- *Examples:*
 - ✓ IRCTC

- ✓ MakeMyTrip
- ✓ Airline booking systems

3. E-Commerce & Online Shopping

- Product catalog, orders, payments, user profiles, reviews
- Databases process millions of transactions
- *Examples:*
 - ✓ Amazon
 - ✓ Flipkart
 - ✓ eBay

4. Education & Student Information Systems

- Student records, marks, attendance, exams, fees
- Centralized database for institutions
- *Examples:*
 - ✓ University ERP systems

- ✓ Online exam portals

5. Healthcare and Hospital Management

- Patient records, billing, appointments, test reports, insurance
- Electronic Medical Records (EMR)
- *Examples:*

- ✓ Hospital Information Systems

- ✓ Lab management systems

6. Telecommunications

- Call records, billing, customer profiles, network usage
- Real-time large database usage
- *Examples:*
- ✓ Jio, Airtel customer billing

7. Social Media Platforms

- User accounts, posts, messages, media storage

- Need efficient databases to handle huge data
- *Examples:*
 - ✓ Facebook
 - ✓ Instagram
 - ✓ Twitter

8. Inventory and Supply Chain Management

- Stock levels, purchase orders, warehouse management, logistics
- Helps avoid overstock/shortage
- *Examples:*
 - ✓ Retail stores
 - ✓ Manufacturing companies

Applications of Database Systems

9. Government & Public Sector

Aadhaar, PAN, voter ID, passport, taxation
Database ensures secure citizen data handling

10. Human Resource Management Systems (HRMS)**

Employee records, salary payroll, attendance
Used widely in corporate companies

11. Scientific Research & Big Data Analytics

Large datasets for AI/ML, simulations, climate data
Databases used for structured data

12. Banking & E-Wallets

Wallet balance, payments, transactions Example:
Paytm, Google Pay

Database Systems versus File Systems

- One way to keep the information on a computer is to store it in operating system files. **File-processing system** is supported by a conventional operating system.
- The system stores permanent records in various files, and it needs different application programs to extract records from, and add records to, the appropriate files.
- **A file management system** is an abstraction to store, retrieve, and management and update a set of files. A File Management System keep track on the files and also manage them

Drawbacks of using file systems

Data redundancy and inconsistency

Multiple file formats, duplication of information in different files

Difficulty in accessing data

Need to write a new program to carry out each new task

Data isolation

multiple files and formats

Database Systems versus File Systems

Integrity problems

- Integrity constraints (e.g. account balance > 0) become part of program code.
- Hard to add new constraints or change existing ones

Atomicity of updates

- Failures may leave database in an inconsistent state with partial updates carried out

E.g. transfer of funds from one account to another should either complete or not happen at all

Concurrent access by multiple users

- Concurrent access needed for performance
- Uncontrolled concurrent accesses can lead to inconsistencies

E.g. two people reading a balance and updating it at the same time

Security problems

Advantages of File Management Systems

1. Simple to Understand and Use

- Easy to design and manage.
- Suitable for small applications.
- Example: maintaining student attendance in a text file

2. Low Cost

- Does not require specialized database software.
- No expensive hardware or administration.

3. Less Resource Intensive

- Consumes less memory and processing power compared to DBMS.
- Can work on low-end machines.

4. Good for Small Data Storage

- Works well where the volume of data is small.
- Ideal for standalone applications.

5. No Need for Trained Personnel

- Users do not need expertise like SQL.
- Basic computer knowledge is sufficient.

6. Fast for Simple Operations

- Direct access to files for storing and retrieving simple data

Disadvantages of File Management Systems

1. Data Redundancy

- Data may be duplicated in multiple files.
- Leads to wastage of storage.
- Example: student name repeated in many files.

2. Data Inconsistency

Due to redundancy, data may become mismatched or outdated.

3. Limited Security

No proper authentication or access control. Anyone can access or modify data

4. No Data Integrity

Cannot enforce rules like primary key, foreign key, validation. Incorrect or inconsistent data may be stored.

5. Difficult Data Sharing

Sharing files among multiple users is hard and error-prone.

Disadvantages of File Management Systems

6. Poor Backup and Recovery

Recovery after system failure is manual and time-consuming.

7. No Concurrency Control

Multiple users cannot update files simultaneously safely.

8. Lack of Standard Query Language

No SQL support.

Searching data becomes slow and complex.

9. Limited Scalability

Cannot handle large volumes of data efficiently.

10. Data Dependency

Application programs depend on file structure. Changing file structure requires changing programs.

Database Systems versus File Systems

Feature	File Management System	Database Management System (DBMS)
Data Storage	Data stored in separate files	Data stored in a centralized database
Data Redundancy	High redundancy (duplicate data)	Low redundancy due to normalization
Data Consistency	Difficult to maintain	Easy to maintain consistency
Security	Limited security	High security with access control, encryption
Data Sharing	Hard to share among users	Multiple users can access simultaneously
Backup & Recovery	Manual backup	Automatic backup and recovery

Database Systems versus File Systems

Data Integrity	No constraints	Constraints like PK, FK, Check ensure integrity
----------------	----------------	---

Database Systems versus File Systems

Feature	File Management System	Database Management System (DBMS)
Concurrency Control	Limited or no concurrency	Multiple users access using locking
Query Language	No standard querying	Uses SQL
Data Independence	No independence	Logical and physical independence
Cost	Cheaper, simple	More expensive and complex
Transaction Control	No transaction management	Supports ACID properties

Database Systems versus File Systems

Storage Structure	Unstructured semi-structured files	or	Structured storage	relational
-------------------	--	----	-----------------------	------------

Data Abstraction:

A database system is a collection of interrelated files and a set of programs that allow users to access and modify these files. It provides users with an *abstract* view of the data. That is, the system hides certain details of how the data are stored and maintained.

Developers hide the complexity levels of from users through several abstraction, to simplify users' interactions with the system.

The data in the DBMS is described at three levels of abstraction

- ❖ Physical Level (Internal Schema)
- ❖ Logical Level (Conceptual Schema)
- ❖ View Level (External Schema)

Physical Level

The lowest level of abstraction describes ***how the data*** are stored. The physical level has an **internal schema** which describes the physical storage structure of the database.

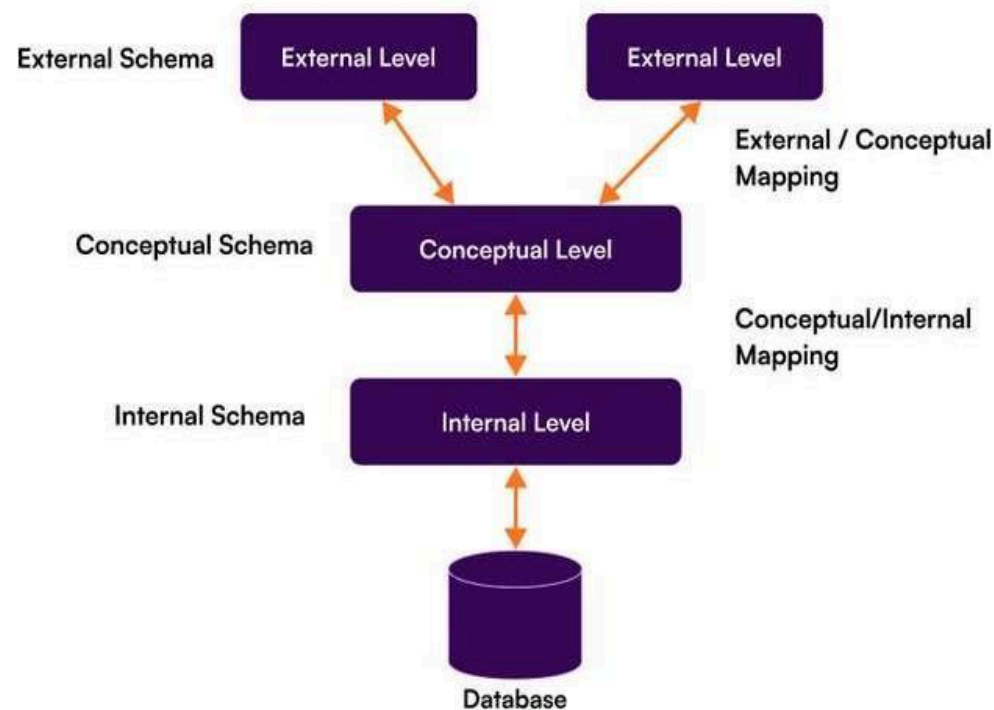
Logical Level

- Logical level describes *what data* are stored in the database, and **what relationships** exist among those data.
- Logical level has a **conceptual schema**, which describes the structure of the whole database for a community of users.

The conceptual schema hides the details of physical storage structures and concentrates on describing entities, data types, relationships, user operations, and constraints.

View level

- This is the highest level in data abstraction.
- This level includes number of external schemas or user views. Each external schema describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group.



Data Independence

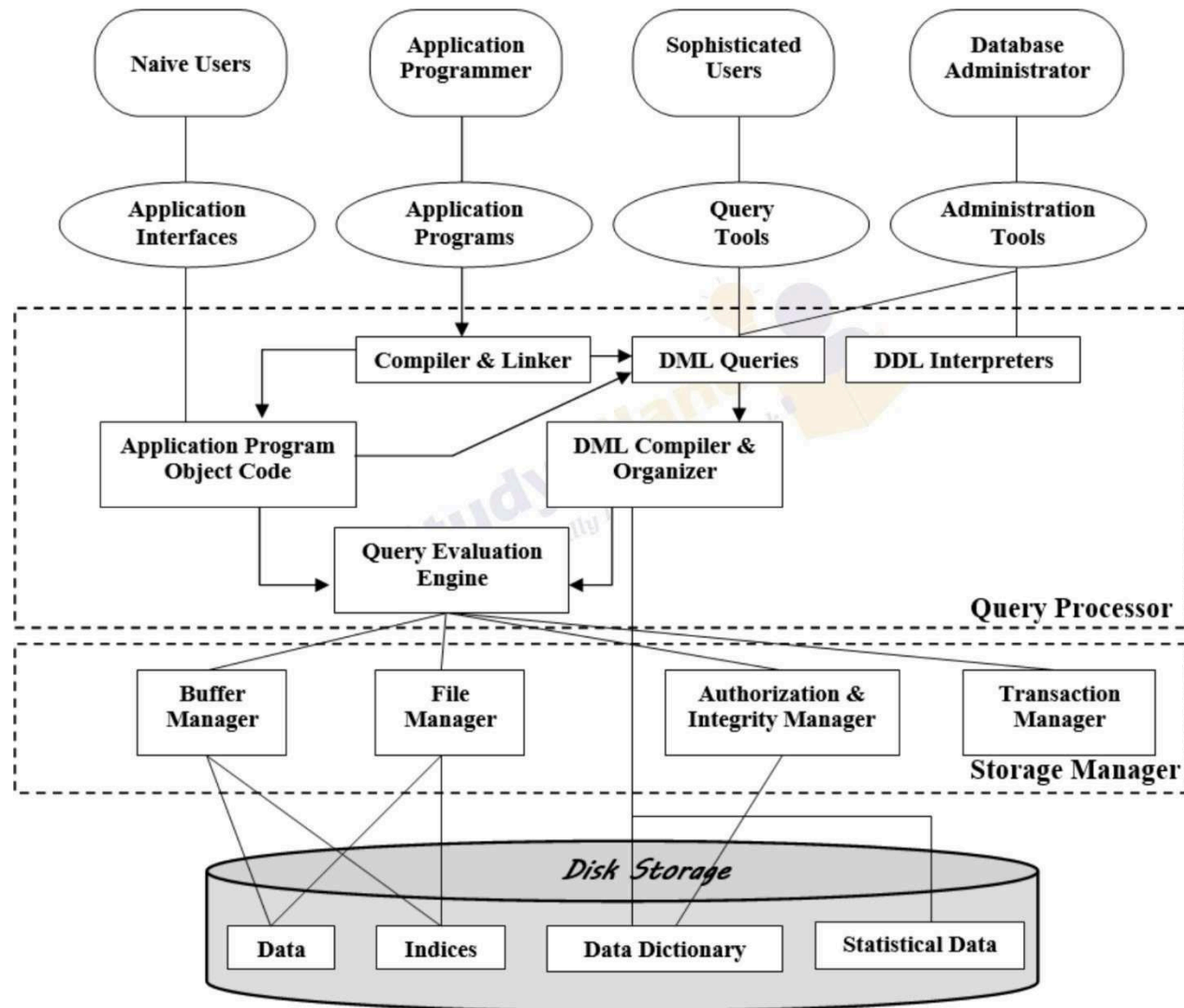
- Data Independence can be defined as the capacity to change the schema at one level of a database system without having to change the schema at the next higher level.
- Metadata itself follows a layered architecture, so that when we change data at one layer, it does not affect the data at another level. This data is independent but mapped to each other.
- There are two types of Data Independence
 - ❖ Logical Data Independence
 - ❖ Physical Data Independence

Logical data independence is the capacity to change the conceptual schema without having to change external schemas or application programs.

Physical data independence is the capacity to change the internal schema without having to change the conceptual schema. Hence, the external schemas need not be changed as well.

Database System Structure

A database system is partitioned into modules that deal with each of the responsibilities of the overall system. The functional components of a database system can be broadly divided into the storage manager and the query processor components.



Database System Structure

- The typical structure of DBMS is based on Relational data model.
- The **top part** of the architecture shows application interfaces used by naive users, application programs created by application programmers, query tools used by sophisticated users and administration tools used by database administrator.
- The **lowest part** of the architecture is for disk storage.
- The **Middle two parts** (Query processor and storage manager) are important components of database architecture.

Storage manager

- A *storage manager* is a program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.
- The storage manager is responsible for storing, retrieving, and updating data in the database. The storage manager components include:
 - **Authorization and integrity manager:** which tests for the satisfaction of integrity constraints and checks the authority of users to access data.
 - **Transaction manager:** which ensures that the database remains in a consistent (correct) state despite system failures, and that concurrent transaction executions proceed without conflicting.
 - **File manager:** which manages the allocation of space on disk storage and the data structures used to represent information stored on disk.
 - **Buffer manager:** which is responsible for fetching data from disk storage into main memory, and deciding what data to cache in main memory. The buffer manager is a critical part of the database system, since it enables the database to handle data sizes that are much larger than the size of main memory.

The storage manager implements several data structures as part of the physical system implementation:

- **Data files:** which store the database itself.
- **Data dictionary:** which stores metadata about the structure of the database, in particular the schema of the database.
- **Indices:** which provide fast access to data items that hold particular values.

The Query Processor

The query processor components include

- **DML compiler**, which translates DML statements in a query language into an evaluation plan consisting of low-level instructions that the query evaluation engine understands.

A query can usually be translated into any of a number of alternative evaluation plans that all give the same result. The DML compiler also performs **query optimization**, that is, it picks the lowest cost evaluation plan from among the alternatives.

- **DDL interpreter**, which interprets DDL statements and records the definitions in the data dictionary.
- **Query evaluation engine**, which executes low-level instructions generated by the DML compiler

Database Users :

In DBMS, users are classified based on how they interact with the database. Each type of user has different technical knowledge and responsibilities

- Naive Users (End Users)
- Application Programmers
- Sophisticated Users
- Specialized Users
- Database Administrator (DBA)
- System Analysts
- Database Designers

1. Naive Users (End Users)

- Also called **casual users**.
- Use the database occasionally for simple tasks.
- Do **not** know database structure or query language.
- Interact through **menus, forms, applications**.
- **Example:** Students checking exam results on a college portal.

2. Application Programmers

- Write **application programs** that interact with the database.
- Use languages such as **Java, Python, C#, PHP** with SQL embedded.
- Responsible for creating front-end applications.
- **Example:** A developer writing a banking app that runs SQL queries internally.

3. Sophisticated Users

- Directly interact with DB using **SQL** (no application).
- Knowledge of DBMS, query languages, and tools.
- Often write their own queries and reports.
- **Example:** Data analysts, business analysts.

4. Specialized Users

- Write **special database applications** beyond standard SQL.
- Work on applications like **AI, CAD systems, scientific databases**.
- Need deep knowledge of DB design & programming.
- **Example:** Developers building a hospital decision support system.

5. Database Administrator (DBA)

The most powerful user of the database.

Responsibilities include:

- Installing and configuring DBMS
- User management (creating roles, granting permissions)
- Security and authorization
- Backup & recovery
- Performance tuning
- Monitoring and troubleshooting

Example: A DBA managing an enterprise-level Oracle or MySQL server.

6. System Analysts

- Understand business requirements.
- Design the overall **database architecture**.
- Act as a bridge between users and developers.

7. Database Designers

- Responsible for **logical and physical design** of the database.
- Create ER diagrams, schemas, constraints, normalization, etc.